09_Assignment_Yuliia_Chernysheva

October 24, 2026

Yuliia Chernysheva
October 24, 2026
CSC 2267

1 CIS 2267 - Assignment 9 - Analyzing Data

1.1 Context

Business companies often run their operations based on information and processes of their Enterprise Resources Planning (ERP) computer systems. Two of the best-known ERP vendors are SAP and Oracle. Companies choose few or many modules depending on their needs and budget. ERP systems are relational databases. As such, SQL is typically used to extract relevant information to run operations and to make decisions.

Company internal data is considered proprietary and is made available to employees on a need-to-know basis. Generally people external to the company have no legal or ethical access to the data. That said, other companies in the company's supply chain might have limited access to coordinate the two companies' activities, such as production planning.

Companies should consider their data as strategic assets, giving them a competitive advantage, and safeguard them accordingly. Nevertheless, some companies are negligent in their computer security protocols. State-sponsored and criminal hackers continually seek to break in and steal any data they can. Guarding against such attacks is an ongoing and expensive challenge.

Such internal datasets are seldom legally/ethically available to outsiders such as ourselves, as we have no legitimate need to know. *Accordingly, we will be using fake data for this project.*

1.2 Situation

You are a training specialist working for the RVN Company, a maker and installer of solar panels. Your company is located in the United Kingdom (UK). Your company records its transactions in British Pounds Sterling, the money of the UK. The three-letter ISO currency code for the British Pound Sterling (usually shortened to "pound") is GBP. The symbol is £. As of April 2022, its value against the US dollar (USD and $) is 1 GBP = 1.31 USD

Your company maintains an ERP system, from which useful information is extracted via SQL queries. As part of the onboarding process for new employees, you will be using this extracted data to train them on how to create pivot tables using Python. **EXCEPT** you will not be using real corporate data for security reasons. **INSTEAD** you will be using specifically developed training (i.e. fake) data, simplified to facilitate learning.

NOTE: Because the data is fake and not intended to be representative of real sales performance, please ignore trying to interpret what the numbers are telling you about our sales performance. Concentrate instead on the mechanics and coding of creating pivot tables.

1.3 Your Dataset

[1]:
```
import pandas as pd
sales = pd.read_csv("sales.csv")
sales
```

[1]:

	Transaction_ID	Year	Region	Type	Product	Quantity_Sold	Price_GBP	Total_Sales_GBP
0	1001	2026	North	Residential	Economy	1020	2000	2040000
1	1002	2026	North	Residential	Good	1035	3000	3105000
2	1003	2026	North	Commercial	Better	1050	4000	4200000
3	1004	2026	North	Commercial	Best	1065	5000	5325000
4	1005	2026	South	Residential	Economy	1080	2200	2376000
..	…	…	…	…	…	…	…	…
59	1060	2027	East	Commercial	Best	1905	4800	9144000
60	1061	2027	West	Residential	Economy	1920	2280	4377600
61	1062	2027	West	Residential	Good	1935	3420	6617700
62	1063	2027	West	Commercial	Better	1950	4560	8892000
63	1064	2027	West	Commercial	Best	1965	5700	11200500

[64 rows x 8 columns]

Independent Variables:
Year can take on 2 values: 2026 and 2027
Region can take on 4 values: North, South, East and West
Type can take on 2 values: Residential and Commercial
Product can take on 4 values: Economy, Good, Better, Best
2 times 4 times 2 times 4 = **64** individual data points.

Value Variables:
Quantity_Sold
Price_GBP

Total_Sales_GBP

Quantity_Sold × Price_GBP = Total_Sales_GBP

With so many independent variables and value variables, groups, aggregates, and pivot tables give us easy ways to arrange the data along different axes.

Group the data by year and calculate the means of the value variables:

[2]:
```
sales.groupby("Year").mean()
```

[2]:

Year	Transaction_ID	Quantity_Sold	Price_GBP	Total_Sales_GBP
2026	1016.5	1252.5	3683.750	4659881.250
2027	1048.5	1732.5	4046.875	7024884.375

Group the data by other independent variables.

[3]:
```
sales.groupby("Region").mean()
```

[3]:

Region	Transaction_ID	Year	Quantity_Sold	Price_GBP	Total_Sales_GBP
East	1034.5	2026.5	1522.5	3368.75	5181618.75
North	1026.5	2026.5	1402.5	3780.00	5412000.00
South	1030.5	2026.5	1462.5	4243.75	6301668.75
West	1038.5	2026.5	1582.5	4068.75	6474243.75

[4]:
```
sales.groupby("Product").mean()
```

[4]:

Product	Transaction_ID	Year	Quantity_Sold	Price_GBP	Total_Sales_GBP
Best	1034.0	2026.5	1515.0	5521.875	8440921.875
Better	1033.0	2026.5	1500.0	4417.500	6686475.000
Economy	1031.0	2026.5	1470.0	2208.750	3276975.000
Good	1032.0	2026.5	1485.0	3313.125	4965159.375

Make a Pandas `DataFrameGroupBy` object, and apply aggregate functions.

[5]:
```
sales_group = sales.groupby(['Year','Region','Product']) \
    [['Transaction_ID','Quantity_Sold','Price_GBP','Total_Sales_GBP']]
sales_group.agg(['sum','mean','min','max'])
```

[5]:

			Transaction_ID				Quantity_Sold	
			sum	mean	min	max	sum	mean
Year	Region	Product						
2026	East	Best	2040	1020.0	1012	1028	2610	1305.0
		Better	2038	1019.0	1011	1027	2580	1290.0
		Economy	2034	1017.0	1009	1025	2520	1260.0
		Good	2036	1018.0	1010	1026	2550	1275.0
	North	Best	2024	1012.0	1004	1020	2370	1185.0

		Better	2022	1011.0	1003	1019	2340	1170.0
		Economy	2018	1009.0	1001	1017	2280	1140.0
		Good	2020	1010.0	1002	1018	2310	1155.0
	South	Best	2032	1016.0	1008	1024	2490	1245.0
		Better	2030	1015.0	1007	1023	2460	1230.0
		Economy	2026	1013.0	1005	1021	2400	1200.0
		Good	2028	1014.0	1006	1022	2430	1215.0
	West	Best	2048	1024.0	1016	1032	2730	1365.0
		Better	2046	1023.0	1015	1031	2700	1350.0
		Economy	2042	1021.0	1013	1029	2640	1320.0
		Good	2044	1022.0	1014	1030	2670	1335.0
2027	East	Best	2104	1052.0	1044	1060	3570	1785.0
		Better	2102	1051.0	1043	1059	3540	1770.0
		Economy	2098	1049.0	1041	1057	3480	1740.0
		Good	2100	1050.0	1042	1058	3510	1755.0
	North	Best	2088	1044.0	1036	1052	3330	1665.0
		Better	2086	1043.0	1035	1051	3300	1650.0
		Economy	2082	1041.0	1033	1049	3240	1620.0
		Good	2084	1042.0	1034	1050	3270	1635.0
	South	Best	2096	1048.0	1040	1056	3450	1725.0
		Better	2094	1047.0	1039	1055	3420	1710.0
		Economy	2090	1045.0	1037	1053	3360	1680.0
		Good	2092	1046.0	1038	1054	3390	1695.0
	West	Best	2112	1056.0	1048	1064	3690	1845.0
		Better	2110	1055.0	1047	1063	3660	1830.0
		Economy	2106	1053.0	1045	1061	3600	1800.0
		Good	2108	1054.0	1046	1062	3630	1815.0

			Price_GBP				Total_Sales_GBP	\
			min	max	sum	mean	min	max	sum
Year	Region	Product							
2026	East	Best	1185	1425	9100	4550.0	4500	4600	11887500
		Better	1170	1410	7280	3640.0	3600	3680	9400800
		Economy	1140	1380	3640	1820.0	1800	1840	4591200
		Good	1155	1395	5460	2730.0	2700	2760	6968700
	North	Best	1065	1305	9750	4875.0	4750	5000	11523750
		Better	1050	1290	7800	3900.0	3800	4000	9102000
		Economy	1020	1260	3900	1950.0	1900	2000	4434000
		Good	1035	1275	5850	2925.0	2850	3000	6738750
	South	Best	1125	1365	11500	5750.0	5500	6000	14377500
		Better	1110	1350	9200	4600.0	4400	4800	11364000
		Economy	1080	1320	4600	2300.0	2200	2400	5544000
		Good	1095	1335	6900	3450.0	3300	3600	8419500
	West	Best	1245	1485	11750	5875.0	5250	6500	16188750
		Better	1230	1470	9400	4700.0	4200	5200	12810000
		Economy	1200	1440	4700	2350.0	2100	2600	6264000
		Good	1215	1455	7050	3525.0	3150	3900	9501750

2027	East	Best	1665	1905	10150	5075.0	4800	5350	18051750
		Better	1650	1890	8120	4060.0	3840	4280	14319600
		Economy	1620	1860	4060	2030.0	1920	2140	7038000
		Good	1635	1875	6090	3045.0	2880	3210	10648350
	North	Best	1545	1785	11850	5925.0	5750	6100	19772250
		Better	1530	1770	9480	4740.0	4600	4880	15675600
		Economy	1500	1740	4740	2370.0	2300	2440	7695600
		Good	1515	1755	7110	3555.0	3450	3660	11650050
	South	Best	1605	1845	12750	6375.0	6150	6600	22047750
		Better	1590	1830	10200	5100.0	4920	5280	17485200
		Economy	1560	1800	5100	2550.0	2460	2640	8589600
		Good	1575	1815	7650	3825.0	3690	3960	12999150
	West	Best	1725	1965	11500	5750.0	5700	5800	21205500
		Better	1710	1950	9200	4600.0	4560	4640	16826400
		Economy	1680	1920	4600	2300.0	2280	2320	8275200
		Good	1695	1935	6900	3450.0	3420	3480	12516300

			mean	min	max
Year	Region	Product			
2026	East	Best	5943750.0	5332500	6555000
		Better	4700400.0	4212000	5188800
		Economy	2295600.0	2052000	2539200
		Good	3484350.0	3118500	3850200
	North	Best	5761875.0	5325000	6198750
		Better	4551000.0	4200000	4902000
		Economy	2217000.0	2040000	2394000
		Good	3369375.0	3105000	3633750
	South	Best	7188750.0	6187500	8190000
		Better	5682000.0	4884000	6480000
		Economy	2772000.0	2376000	3168000
		Good	4209750.0	3613500	4806000
	West	Best	8094375.0	6536250	9652500
		Better	6405000.0	5166000	7644000
		Economy	3132000.0	2520000	3744000
		Good	4750875.0	3827250	5674500
2027	East	Best	9025875.0	8907750	9144000
		Better	7159800.0	7062000	7257600
		Economy	3519000.0	3466800	3571200
		Good	5324175.0	5248350	5400000
	North	Best	9886125.0	8883750	10888500
		Better	7837800.0	7038000	8637600
		Economy	3847800.0	3450000	4245600
		Good	5825025.0	5226750	6423300
	South	Best	11023875.0	9870750	12177000
		Better	8742600.0	7822800	9662400
		Economy	4294800.0	3837600	4752000

```
              Good      6499575.0   5811750   7187400
      West    Best     10602750.0  10005000  11200500
              Better    8413200.0   7934400   8892000
              Economy   4137600.0   3897600   4377600
              Good      6258150.0   5898600   6617700
```

Make a pivot table, showing years for the rows, showing the regions in columns and quantity sold and price as the values.

[6]:
```
sales.pivot_table(
    index='Year',
    columns='Region',
    values=['Price_GBP','Quantity_Sold'],
    aggfunc='mean'
)
```

[6]:

	Price_GBP				Quantity_Sold			
Region	East	North	South	West	East	North	South	West
Year								
2026	3185.0	3412.5	4025.0	4112.5	1282.5	1162.5	1222.5	1342.5
2027	3552.5	4147.5	4462.5	4025.0	1762.5	1642.5	1702.5	1822.5

Make several pivot table using different rows, columns, and values.

[7]:
```
sales.pivot_table(
    index='Region',
    columns='Year',
    values=['Price_GBP','Quantity_Sold'],
    aggfunc='mean'
)
```

[7]:

	Price_GBP		Quantity_Sold	
Year	2026	2027	2026	2027
Region				
East	3185.0	3552.5	1282.5	1762.5
North	3412.5	4147.5	1162.5	1642.5
South	4025.0	4462.5	1222.5	1702.5
West	4112.5	4025.0	1342.5	1822.5

[8]:
```
sales.pivot_table(
    index='Type',
    columns='Region',
    values='Price_GBP',
    aggfunc='mean'
)
```

[8]:
```
Region          East   North    South     West
Type
Commercial   4331.25  4860.0  5456.25  5231.25
Residential  2406.25  2700.0  3031.25  2906.25
```

[9]:
```
sales.pivot_table(
    index='Region',
    columns='Type',
    values='Total_Sales_GBP',
    aggfunc=['min','max']
)
```

[9]:
```
              min                    max
Type   Commercial Residential Commercial Residential
Region
East      4212000     2052000    9144000     5400000
North     4200000     2040000   10888500     6423300
South     4884000     2376000   12177000     7187400
West      5166000     2520000   11200500     6617700
```

[10]:
```
sales.pivot_table(
    index='Year',
    columns='Product',
    values=['Price_GBP','Total_Sales_GBP'],
    aggfunc=['min','max']
)
```

[10]:
```
              min                                                    \
        Price_GBP                     Total_Sales_GBP
Product      Best Better Economy  Good            Best   Better  Economy
Year
2026         4500   3600    1800  2700         5325000  4200000  2040000
2027         4800   3840    1920  2880         8883750  7038000  3450000

                      max                                                \
                Price_GBP                     Total_Sales_GBP
Product     Good      Best Better Economy  Good            Best   Better
Year
2026     3105000      6500   5200    2600  3900         9652500  7644000
2027     5226750      6600   5280    2640  3960        12177000  9662400



Product  Economy     Good
Year
2026     3744000  5674500
2027     4752000  7187400
```

[11]:
```
sales.pivot_table(
    index=['Year','Product'],
    columns='Region',
    values=['Price_GBP','Quantity_Sold'],
    aggfunc='mean'
)
```

[11]:

		Price_GBP				Quantity_Sold			
Region		East	North	South	West	East	North	South	West
Year	Product								
2026	Best	4550	4875	5750	5875	1305	1185	1245	1365
	Better	3640	3900	4600	4700	1290	1170	1230	1350
	Economy	1820	1950	2300	2350	1260	1140	1200	1320
	Good	2730	2925	3450	3525	1275	1155	1215	1335
2027	Best	5075	5925	6375	5750	1785	1665	1725	1845
	Better	4060	4740	5100	4600	1770	1650	1710	1830
	Economy	2030	2370	2550	2300	1740	1620	1680	1800
	Good	3045	3555	3825	3450	1755	1635	1695	1815

Calculate the percent change of total sales, grouped by year, then region.

[12]:
```
df = sales.pivot_table(
    index=['Year','Region'],
    values='Total_Sales_GBP',
    aggfunc='sum'
)
df.pct_change()
```

[12]:

		Total_Sales_GBP
Year	Region	
2026	East	NaN
	North	-0.031956
	South	0.248644
	West	0.127427
2027	East	0.118245
	North	0.094607
	South	0.115492
	West	-0.037602

[ ]:
```

```